

Nagyhazi

1.0

Generated by Doxygen 1.15.0

1 Directory Hierarchy	1
1.1 Directories	1
2 Data Structure Index	3
2.1 Data Structures	3
3 File Index	5
3.1 File List	5
4 Directory Documentation	7
4.1 D:/bme/prog1/nagyhazi/src Directory Reference	7
5 Data Structure Documentation	9
5.1 Node Struct Reference	9
5.1.1 Detailed Description	9
5.1.2 Field Documentation	9
5.1.2.1 character	9
5.1.2.2 dash	9
5.1.2.3 dot	10
5.2 Split_str Struct Reference	10
5.2.1 Detailed Description	10
5.2.2 Field Documentation	10
5.2.2.1 arr	10
5.2.2.2 len	10
6 File Documentation	11
6.1 D:/bme/prog1/nagyhazi/src/file_handling.c File Reference	11
6.1.1 Function Documentation	11
6.1.1.1 file_to_morse_tree()	11
6.1.1.2 read_all_lines()	11
6.1.1.3 remove_new_line_chars()	12
6.1.1.4 write_to_file()	12
6.2 file_handling.c	13
6.3 D:/bme/prog1/nagyhazi/src/file_handling.h File Reference	14
6.3.1 Function Documentation	14
6.3.1.1 file_to_morse_tree()	14
6.3.1.2 read_all_lines()	15
6.3.1.3 remove_new_line_chars()	15
6.3.1.4 write_to_file()	16
6.4 file_handling.h	16
6.5 D:/bme/prog1/nagyhazi/src/main.c File Reference	17
6.5.1 Function Documentation	17
6.5.1.1 main()	17
6.6 main.c	17

6.7 D:/bme/prog1/nagyhazi/src/morse.c File Reference	18
6.7.1 Function Documentation	19
6.7.1.1 add_generate_morse()	19
6.7.1.2 decode_morse_char()	19
6.7.1.3 decode_morse_text()	19
6.7.1.4 decode_morse_word()	20
6.7.1.5 dump_node()	20
6.7.1.6 dump_tree()	20
6.7.1.7 encode_char()	21
6.7.1.8 encode_text()	21
6.7.1.9 encode_word()	21
6.7.1.10 free_tree()	22
6.7.1.11 init_node()	22
6.7.1.12 print_error_and_exit()	22
6.7.1.13 try_add_node()	23
6.8 morse.c	23
6.9 D:/bme/prog1/nagyhazi/src/morse.h File Reference	25
6.9.1 Typedef Documentation	26
6.9.1.1 Node	26
6.9.2 Function Documentation	26
6.9.2.1 add_generate_morse()	26
6.9.2.2 decode_morse_char()	27
6.9.2.3 decode_morse_text()	27
6.9.2.4 decode_morse_word()	28
6.9.2.5 dump_node()	29
6.9.2.6 dump_tree()	29
6.9.2.7 encode_char()	30
6.9.2.8 encode_text()	30
6.9.2.9 encode_word()	31
6.9.2.10 free_tree()	32
6.9.2.11 init_node()	32
6.9.2.12 print_error_and_exit()	33
6.9.2.13 try_add_node()	33
6.10 morse.h	34
6.11 D:/bme/prog1/nagyhazi/src/split_str.c File Reference	34
6.11.1 Function Documentation	35
6.11.1.1 flatten_split_str()	35
6.11.1.2 free_split_str()	35
6.11.1.3 split_str()	35
6.12 split_str.c	36
6.13 D:/bme/prog1/nagyhazi/src/split_str.h File Reference	37
6.13.1 Typedef Documentation	37

6.13.1.1 Split_str	37
6.13.2 Function Documentation	37
6.13.2.1 flatten_split_str()	37
6.13.2.2 free_split_str()	38
6.13.2.3 split_str()	38
6.14 split_str.h	39
6.15 D:/bme/prog1/nagyhazi/src/tests.c File Reference	39
6.15.1 Function Documentation	40
6.15.1.1 mock_split_str()	40
6.15.1.2 mock_tree()	40
6.15.1.3 run_all_tests()	40
6.15.1.4 test_add_generate_morse()	40
6.15.1.5 test_decode_char()	40
6.15.1.6 test_decode_text()	41
6.15.1.7 test_decode_word()	41
6.15.1.8 test_encode_char()	41
6.15.1.9 test_encode_text()	41
6.15.1.10 test_encode_word()	41
6.15.1.11 test_file_to_morse_tree()	41
6.15.1.12 test_flatten_split_str()	41
6.15.1.13 test_free_split_str()	42
6.15.1.14 test_free_tree()	42
6.15.1.15 test_init_node()	42
6.15.1.16 test_read_all_lines()	42
6.15.1.17 test_remove_new_lines_chars()	42
6.15.1.18 test_split_str()	42
6.15.1.19 test_write_to_file()	42
6.16 tests.c	43
6.17 D:/bme/prog1/nagyhazi/src/tests.h File Reference	46
6.17.1 Function Documentation	46
6.17.1.1 mock_split_str()	46
6.17.1.2 mock_tree()	47
6.17.1.3 run_all_tests()	47
6.17.1.4 test_add_generate_morse()	47
6.17.1.5 test_decode_char()	47
6.17.1.6 test_decode_text()	47
6.17.1.7 test_decode_word()	47
6.17.1.8 test_encode_char()	47
6.17.1.9 test_encode_text()	48
6.17.1.10 test_encode_word()	48
6.17.1.11 test_file_to_morse_tree()	48
6.17.1.12 test_flatten_split_str()	48

6.17.1.13 test_free_split_str()	48
6.17.1.14 test_free_tree()	48
6.17.1.15 test_init_node()	48
6.17.1.16 test_read_all_lines()	49
6.17.1.17 test_remove_new_lines_chars()	49
6.17.1.18 test_split_str()	49
6.17.1.19 test_write_to_file()	49
6.18 tests.h	49
Index	51

Chapter 1

Directory Hierarchy

1.1 Directories

src	7
file_handling.c	11
file_handling.h	14
main.c	17
morse.c	18
morse.h	25
split_str.c	34
split_str.h	37
tests.c	39
tests.h	46

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Node		9
Split_str		10

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

D:/bme/prog1/nagyhazi/src/ file_handling.c	11
D:/bme/prog1/nagyhazi/src/ file_handling.h	14
D:/bme/prog1/nagyhazi/src/ main.c	17
D:/bme/prog1/nagyhazi/src/ morse.c	18
D:/bme/prog1/nagyhazi/src/ morse.h	25
D:/bme/prog1/nagyhazi/src/ split_str.c	34
D:/bme/prog1/nagyhazi/src/ split_str.h	37
D:/bme/prog1/nagyhazi/src/ tests.c	39
D:/bme/prog1/nagyhazi/src/ tests.h	46

Chapter 4

Directory Documentation

4.1 D:/bme/prog1/nagyhazi/src Directory Reference

Files

- file [file_handling.c](#)
- file [file_handling.h](#)
- file [main.c](#)
- file [morse.c](#)
- file [morse.h](#)
- file [split_str.c](#)
- file [split_str.h](#)
- file [tests.c](#)
- file [tests.h](#)

Chapter 5

Data Structure Documentation

5.1 Node Struct Reference

```
#include <morse.h>
```

Data Fields

- char [character](#)
- struct [Node](#) * [dot](#)
- struct [Node](#) * [dash](#)

5.1.1 Detailed Description

Definition at line [18](#) of file [morse.h](#).

5.1.2 Field Documentation

5.1.2.1 character

```
char character
```

Definition at line [19](#) of file [morse.h](#).

5.1.2.2 dash

```
struct Node* dash
```

Definition at line [21](#) of file [morse.h](#).

5.1.2.3 dot

```
struct Node* dot
```

Definition at line 20 of file [morse.h](#).

The documentation for this struct was generated from the following file:

- D:/bme/prog1/nagyhazi/src/[morse.h](#)

5.2 Split_str Struct Reference

```
#include <split_str.h>
```

Data Fields

- int [len](#)
- char ** [arr](#)

5.2.1 Detailed Description

Definition at line 15 of file [split_str.h](#).

5.2.2 Field Documentation

5.2.2.1 arr

```
char** arr
```

Definition at line 17 of file [split_str.h](#).

5.2.2.2 len

```
int len
```

Definition at line 16 of file [split_str.h](#).

The documentation for this struct was generated from the following file:

- D:/bme/prog1/nagyhazi/src/[split_str.h](#)

Chapter 6

File Documentation

6.1 D:/bme/prog1/nagyhazi/src/file_handling.c File Reference

```
#include "file_handling.h"
```

Functions

- void [file_to_morse_tree](#) (Node **start, char *filename)
- int [write_to_file](#) (char *text, char *filename)
- void [remove_new_line_chars](#) (char **str)
- char * [read_all_lines](#) (char *filename)

6.1.1 Function Documentation

6.1.1.1 file_to_morse_tree()

```
void file_to_morse_tree (  
    Node ** start,  
    char * filename)
```

Reads morse code definitions from a file and builds a morse code binary tree The file should contain lines with format: character morse_code Each line consists of a single character, a space, and its morse code (dots and dashes) For example: a .- b -.. c -.-.

Parameters

<i>start</i>	Pointer to the root node of the morse tree to be populated
<i>filename</i>	Path to the file containing morse code definitions

Definition at line 8 of file [file_handling.c](#).

6.1.1.2 read_all_lines()

```
char * read_all_lines (  
    char * filename)
```

Parameters

<i>filename</i>	Path to the file to be read
-----------------	-----------------------------

Returns

String containing all file contents, or NULL if file cannot be opened

Definition at line 60 of file [file_handling.c](#).

6.1.1.3 remove_new_line_chars()

```
void remove_new_line_chars (  
    char ** str)
```

Removes newline and carriage return characters from a string Replaces '\n' and '\r' characters with spaces to clean up text Modifies the string in place by creating a new cleaned string

Parameters

<i>str</i>	Pointer to a string pointer to be modified with cleaned content
------------	---

Definition at line 41 of file [file_handling.c](#).

6.1.1.4 write_to_file()

```
int write_to_file (  
    char * text,  
    char * filename)
```

Writes text content to a file, creating the file if it does not exist If the file already exists, the function will exit with an error

Parameters

<i>text</i>	Text content string to be written to the file
<i>filename</i>	Path where the file should be created and written to

Returns

1 if an error occurs, otherwise 0 on success

Definition at line 28 of file [file_handling.c](#).

6.2 file_handling.c

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/13/2025.
00003 //
00004
00005 #include "file_handling.h"
00006
00007 void file_to_morse_tree(Node **start, char *filename) {
00008     char *lines = read_all_lines(filename);
00009     Split_str *arr = split_str(lines, '\n');
00010     free(lines);
00011     for (int i = 0; i < arr->len; ++i) {
00012         Split_str *code = split_str(arr->arr[i], ' ');
00013         if (code->len != 2) {
00014             free_split_str(code);
00015             free_split_str(arr);
00016             free_tree(*start);
00017             free(*start);
00018             print_error_and_exit(3, "Helytelen kodtabla formatum");
00019         }
00020         add_generate_morse(start, code->arr[1], code->arr[0][0]);
00021         free_split_str(code);
00022     }
00023     free_split_str(arr);
00024 }
00025
00026 int write_to_file(char *text, char *filename) {
00027     FILE *checkfp = fopen(filename, "r");
00028     if (checkfp != NULL) {
00029         fclose(checkfp);
00030         return 1;
00031     }
00032     FILE *fptr = fopen(filename, "w");
00033     fprintf(fptr, "%s", text);
00034     fclose(fptr);
00035     return 0;
00036 }
00037
00038 void remove_new_line_chars(char **str) {
00039     int ct = 0;
00040     int ct_no_new_lines = 0;
00041     while ((*str)[ct++] != '\0')
00042         if ((*str)[ct] != '\n')
00043             ++ct_no_new_lines;
00044     int ct_final = 0;
00045     char *final = malloc((ct_no_new_lines + 1) * sizeof(char));
00046     for (int i = 0; i < ct_no_new_lines; ++i)
00047         if ((*str)[ct_no_new_lines] == '\n')
00048             final[ct_final++] = (*str)[ct_no_new_lines];
00049     final[ct_no_new_lines] = '\0';
00050     free(*str);
00051     *str = final;
00052 }
00053
00054 char *read_all_lines(char *filename) {
00055     FILE *fptr = fopen(filename, "r");
00056     if (fptr == NULL) {
00057         fclose(fptr);
00058         print_error_and_exit(6, "Ilyen nevű fájlt nem lehet elérni");
00059     }
00060     int buffersize = 31;
00061     char *buff = malloc(buffersize * sizeof(char));
00062     buff[0] = '\0';
00063     char tmp[31];
00064     while (fgets(tmp, sizeof(tmp), fptr) != NULL) {
00065         if (buffersize + strlen(tmp) + 1 > buffersize) {
00066             buffersize += 31;
00067             char *new_buff = malloc(buffersize * sizeof(char));
00068             strcpy(new_buff, buff);
00069             free(buff);
00070             buff = new_buff;
00071         }
00072         strcat(buff, tmp);
00073     }
00074     int ct = 0;
00075     while (buff[ct] != '\0')
00076         if (buff[ct] != '\r')
00077             ++ct;
00078 }

```

```

00087     int ct_final = 0;
00088     char *final = malloc((ct + 1) * sizeof(char));
00089     for (int i = 0; i < ct; ++i)
00090         if (buff[ct] != '\r')
00091             final[ct_final++] = buff[i];
00092     final[ct] = '\0';
00093
00094     free(buff);
00095     fclose(fp);
00096     return final;
00097 }

```

6.3 D:/bme/prog1/nagyhazi/src/file_handling.h File Reference

```

#include "morse.h"
#include "split_str.h"
#include <stdlib.h>

```

Functions

- void [file_to_morse_tree](#) (Node **start, char *filename)
- int [write_to_file](#) (char *text, char *filename)
- char * [read_all_lines](#) (char *filename)
- void [remove_new_line_chars](#) (char **str)

6.3.1 Function Documentation

6.3.1.1 file_to_morse_tree()

```

void file_to_morse_tree (
    Node ** start,
    char * filename)

```

Reads morse code definitions from a file and builds a morse code binary tree The file should contain lines with format: character morse_code Each line consists of a single character, a space, and its morse code (dots and dashes) For example: a .- b -.. c -.-.

Parameters

<i>start</i>	Pointer to the root node of the morse tree to be populated
<i>filename</i>	Path to the file containing morse code definitions

Reads morse code definitions from a file and builds a morse code binary tree The file should contain lines with format: character morse_code Each line consists of a single character, a space, and its morse code (dots and dashes) For example: a .- b -.. c -.-.

Parameters

<i>start</i>	Pointer to the root node of the morse tree to be populated
--------------	--

<i>filename</i>	Path to the file containing morse code definitions
-----------------	--

Definition at line 8 of file [file_handling.c](#).

6.3.1.2 read_all_lines()

```
char * read_all_lines (
    char * filename)
```

Reads all lines from a file into a single dynamically allocated string Preserves newline characters from the file in the returned string Removes carriage return characters that may be present on Windows systems

Parameters

<i>filename</i>	Path to the file to be read
-----------------	-----------------------------

Returns

String containing all file contents, or NULL if file cannot be opened

Reads all lines from a file into a single dynamically allocated string Preserves newline characters from the file in the returned string Removes carriage return characters that may be present on Windows systems

Parameters

<i>filename</i>	Path to the file to be read
-----------------	-----------------------------

Returns

String containing all file contents, or NULL if file cannot be opened

Definition at line 60 of file [file_handling.c](#).

6.3.1.3 remove_new_line_chars()

```
void remove_new_line_chars (
    char ** str)
```

Removes newline and carriage return characters from a string Replaces '\n' and '\r' characters with spaces to clean up text Modifies the string in place by creating a new cleaned string

Parameters

<i>str</i>	Pointer to a string pointer to be modified with cleaned content
------------	---

Removes newline and carriage return characters from a string Replaces '\n' and '\r' characters with spaces to clean up text Modifies the string in place by creating a new cleaned string

Parameters

<i>str</i>	Pointer to a string pointer to be modified with cleaned content
------------	---

Definition at line 41 of file [file_handling.c](#).

6.3.1.4 write_to_file()

```
int write_to_file (  
    char * text,  
    char * filename)
```

Writes text content to a file, creating the file if it does not exist If the file already exists, the function will exit with an error

Parameters

<i>text</i>	Text content string to be written to the file
<i>filename</i>	Path where the file should be created and written to

Returns

1 if an error occurs, otherwise 0 on success

Writes text content to a file, creating the file if it does not exist If the file already exists, the function will exit with an error

Parameters

<i>text</i>	Text content string to be written to the file
<i>filename</i>	Path where the file should be created and written to

Returns

1 if an error occurs, otherwise 0 on success

Definition at line 28 of file [file_handling.c](#).

6.4 file_handling.h

[Go to the documentation of this file.](#)

```
00001 //  
00002 // Created by User01 on 11/13/2025.  
00003 //  
00004  
00005 #ifndef NAGYHAZI_FILE_HANDLING_H  
00006 #define NAGYHAZI_FILE_HANDLING_H  
00007  
00008 #ifdef DEBUG_MALLOC
```

```

00009 #include "debugmalloc.h"
00010 #endif // DEBUG_MALLOC
00011
00012 #include "morse.h"
00013 #include "split_str.h"
00014
00015 #include <stdlib.h>
00016
00025 void file_to_morse_tree(Node **start, char *filename);
00026
00034 int write_to_file(char *text, char *filename);
00035
00043 char *read_all_lines(char *filename);
00044
00051 void remove_new_line_chars(char **str);
00052
00053 #endif // NAGYHAZI_FILE_HANDLING_H

```

6.5 D:/bme/prog1/nagyhazi/src/main.c File Reference

```

#include "file_handling.h"
#include "morse.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Functions

- int [main](#) (int argc, char **argv)

6.5.1 Function Documentation

6.5.1.1 main()

```

int main (
    int argc,
    char ** argv)

```

Definition at line 16 of file [main.c](#).

6.6 main.c

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/13/2025.
00003 //
00004
00005 #ifdef DEBUG_MALLOC
00006 #include "debugmalloc.h"
00007 #endif // DEBUG_MALLOC
00008
00009 #include "file_handling.h"
00010 #include "morse.h"
00011
00012 #include <stdio.h>
00013 #include <stdlib.h>
00014 #include <string.h>
00015

```

```

00016 int main(int argc, char **argv) {
00017     #ifdef ENABLE_TESTS
00018     #include "tests.h"
00019     run_all_tests();
00020     return 0;
00021 #endif // ENABLE_TESTS
00022
00023     char *out_filename = malloc((strlen("./result.txt") + 1) * sizeof(char));
00024     strcpy(out_filename, "./result.txt");
00025     if (argc > 2) {
00026         free(out_filename);
00027         out_filename = malloc((strlen(argv[argc - 1]) + 1) * sizeof(char));
00028         strcpy(out_filename, argv[argc - 1]);
00029     }
00030
00031     int is_reverse = 0;
00032     if (strcmp(argv[1], "-r") == 0)
00033         is_reverse = 1;
00034
00035     Node *start = init_node();
00036     char *text_content = read_all_lines(argv[2 + is_reverse]);
00037     file_to_morse_tree(&start, argv[1 + is_reverse]);
00038
00039     char *coded_text = is_reverse ? decode_morse_text(start, text_content)
00040                                   : encode_text(start, text_content);
00041
00042     const int write_rtn = write_to_file(coded_text, out_filename);
00043     if (write_rtn == 0)
00044         printf("Sikeres forditas %s -> %s\n", argv[2 + is_reverse], out_filename);
00045
00046     free(coded_text);
00047     free(out_filename);
00048     free(text_content);
00049     free_tree(start);
00050     free(start);
00051
00052     if (write_rtn == 1)
00053         print_error_and_exit(4, "A kimeneti fajl mar letezik");
00054
00055     return 0;
00056 }

```

6.7 D:/bme/prog1/nagyhazi/src/morse.c File Reference

```
#include "morse.h"
```

Functions

- char * [encode_text](#) (Node *start, char *text)
- char * [encode_word](#) (Node *start, char *word)
- char * [encode_char](#) (Node *start, char character)
- char * [decode_morse_text](#) (Node *start, char *morse_text)
- char * [decode_morse_word](#) (Node *start, char *morse_word)
- Node * [init_node](#) (void)
- char [decode_morse_char](#) (Node *start, char *code)
- void [add_generate_morse](#) (Node **start, const char *code, const char character)
- void [try_add_node](#) (Node **node)
- void [free_tree](#) (Node *start)
- void [dump_tree](#) (Node *start)
- void [dump_node](#) (Node *start, int counter)
- void [print_error_and_exit](#) (const int error_code, char *description)

6.7.1 Function Documentation

6.7.1.1 add_generate_morse()

```
void add_generate_morse (
    Node ** start,
    const char * code,
    const char character)
```

Generates morse code nodes in the tree based on a morse code pattern Traverses the tree following the dot (.) and dash (-) characters in the code Creates new nodes as needed and stores the character at the final position

Parameters

<i>start</i>	Pointer to the root node of the morse code tree
<i>code</i>	Morse code pattern string containing dots and dashes
<i>character</i>	Character to be stored at the end of the morse code path

Definition at line 139 of file [morse.c](#).

6.7.1.2 decode_morse_char()

```
char decode_morse_char (
    Node * start,
    char * code)
```

Decodes a single morse code character into its plain text representation Traverses the morse tree following the dot (.) and dash (-) pattern

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>code</i>	Morse code string containing dots and dashes

Returns

Decoded character, or null character if morse code not found in tree

Definition at line 116 of file [morse.c](#).

6.7.1.3 decode_morse_text()

```
char * decode_morse_text (
    Node * start,
    char * morse_text)
```

Decodes a complete morse code text string back into plain text Morse words in the input are separated by tabs and decoded individually The decoded words are joined with spaces in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_text</i>	Morse code text string to be decoded

Returns

Decoded plain text string with words separated by spaces, or NULL on error

Definition at line 82 of file [morse.c](#).

6.7.1.4 decode_morse_word()

```
char * decode_morse_word (  
    Node * start,  
    char * morse_word)
```

Decodes a morse code word back into a plain text word Individual morse characters within the word are separated by spaces

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_word</i>	Morse code word string to be decoded

Returns

Decoded plain text word, or NULL on error

Definition at line 96 of file [morse.c](#).

6.7.1.5 dump_node()

```
void dump_node (  
    Node * start,  
    int counter)
```

Recursively prints morse tree nodes with indentation based on depth Helper function for dump_tree that handles the recursive traversal

Parameters

<i>start</i>	Current node being printed
<i>counter</i>	Indentation level counter for formatting output

Definition at line 184 of file [morse.c](#).

6.7.1.6 dump_tree()

Parameters

<i>start</i>	Root node of the tree to be displayed
--------------	---------------------------------------

Definition at line 181 of file [morse.c](#).

6.7.1.7 encode_char()

```
char * encode_char (  
    Node * start,  
    char character)
```

Encodes a single character into its morse code representation Recursively traverses the morse tree to find the character and returns its path

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>character</i>	Single character to be encoded into morse code

Returns

Morse code string (dots and dashes) representing the character, or NULL if not found

Definition at line 40 of file [morse.c](#).

6.7.1.8 encode_text()

```
char * encode_text (  
    Node * start,  
    char * text)
```

Encodes a complete text string into morse code by encoding each word separately Words in the input text are separated by spaces and encoded individually The encoded words are joined with tab characters in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>text</i>	Text string to be encoded into morse code

Returns

Encoded morse code string with words separated by tabs, or NULL on error

Definition at line 8 of file [morse.c](#).

6.7.1.9 encode_word()

```
char * encode_word (  
    Node * start,
```

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>word</i>	Word string to be encoded into morse code

Returns

Encoded morse code string with characters separated by spaces, or NULL on error

Definition at line 22 of file [morse.c](#).

6.7.1.10 free_tree()

```
void free_tree (  
    Node * start)
```

Recursively frees all memory allocated for a morse code tree Performs depth-first traversal to deallocate all nodes in both branches

Parameters

<i>start</i>	Root node of the morse tree to be freed
--------------	---

Definition at line 171 of file [morse.c](#).

6.7.1.11 init_node()

```
Node * init_node (  
    void )
```

Initializes a new morse tree node with default values Allocates memory for a [Node](#) and sets all fields to empty/NULL values The character is set to null terminator and both pointers are set to NULL

Returns

Pointer to newly allocated and initialized [Node](#) structure

Definition at line 107 of file [morse.c](#).

6.7.1.12 print_error_and_exit()

```
void print_error_and_exit (  
    const int error_code,  
    char * description)
```

Prints an error message to standard output and terminates the program Used for critical errors that prevent program continuation

Parameters

<i>error_code</i>	Numeric error code identifying the error type
<i>description</i>	Human-readable error description message

Definition at line 194 of file [morse.c](#).

6.7.1.13 try_add_node()

```
void try_add_node (
    Node ** node)
```

Attempts to allocate a new node if the target pointer is NULL. If the pointer already points to allocated memory, the function does nothing. Used as a helper to safely create child nodes during tree construction.

Parameters

<i>node</i>	Pointer to a node pointer that may need allocation
-------------	--

Definition at line 164 of file [morse.c](#).

6.8 morse.c

[Go to the documentation of this file.](#)

```
00001 //
00002 // Created by User01 on 11/13/2025.
00003 //
00004
00005 #include "morse.h"
00006
00008 char *encode_text(Node *start, char *text) {
00009     Split_str *arr = split_str(text, ' ');
00010     for (int i = 0; i < arr->len; ++i) {
00011         char *temp = encode_word(start, arr->arr[i]);
00012         free(arr->arr[i]);
00013         arr->arr[i] = temp;
00014     }
00015
00016     char *encoded = flatten_split_str(arr, "\t");
00017     free_split_str(arr);
00018     return encoded;
00019 }
00020
00022 char *encode_word(Node *start, char *word) {
00023     Split_str *encoded = malloc(sizeof(Split_str));
00024     encoded->len = strlen(word);
00025     encoded->arr = malloc(encoded->len * sizeof(char *));
00026     for (int i = 0; i < encoded->len; ++i) {
00027         char *encoded_char = encode_char(start, word[i]);
00028         if (encoded_char == NULL)
00029             print_error_and_exit(5, "Sikertelen forditas hanyos kodtabla miatt");
00030         int encoded_char_len = strlen(encoded_char);
00031         encoded->arr[i] = encoded_char;
00032         encoded->arr[i][encoded_char_len] = '\0';
00033     }
00034     char *flattened_encoded_char = flatten_split_str(encoded, " ");
00035     free_split_str(encoded);
00036     return flattened_encoded_char;
00037 }
00038
00040 char *encode_char(Node *start, char character) {
00041     if (start == NULL)
00042         return NULL;
```

```

00043
00044     if (start->character == character) {
00045         char *base = malloc(1);
00046         if (base == NULL)
00047             return NULL;
00048         base[0] = '\0';
00049         return base;
00050     }
00051
00052     char *dot_path = encode_char(start->dot, character);
00053     if (dot_path != NULL) {
00054         char *res = malloc((strlen(dot_path) + 2) * sizeof(char));
00055         if (res == NULL) {
00056             free(dot_path);
00057             return NULL;
00058         }
00059         res[0] = '.';
00060         strcpy(res + 1, dot_path);
00061         free(dot_path);
00062         return res;
00063     }
00064
00065     char *dash_path = encode_char(start->dash, character);
00066     if (dash_path != NULL) {
00067         char *res = malloc((strlen(dash_path) + 2) * sizeof(char));
00068         if (res == NULL) {
00069             free(dash_path);
00070             return NULL;
00071         }
00072         res[0] = '-';
00073         strcpy(res + 1, dash_path);
00074         free(dash_path);
00075         return res;
00076     }
00077
00078     return NULL;
00079 }
00080
00082 char *decode_morse_text(Node *start, char *morse_text) {
00083     Split_str *arr = split_str(morse_text, '\t');
00084     for (int i = 0; i < arr->len; ++i) {
00085         char *temp = decode_morse_word(start, arr->arr[i]);
00086         free(arr->arr[i]);
00087         arr->arr[i] = temp;
00088     }
00089
00090     char *decoded = flatten_split_str(arr, " ");
00091     free_split_str(arr);
00092     return decoded;
00093 }
00094
00096 char *decode_morse_word(Node *start, char *morse_word) {
00097     Split_str *arr = split_str(morse_word, ' ');
00098     char *decoded = malloc((arr->len + 1) * sizeof(char));
00099     for (int i = 0; i < arr->len; ++i)
00100         decoded[i] = decode_morse_char(start, arr->arr[i]);
00101     decoded[arr->len] = '\0';
00102     free_split_str(arr);
00103     return decoded;
00104 }
00105
00107 Node *init_node(void) {
00108     Node *new = malloc(sizeof(Node));
00109     new->character = '\0';
00110     new->dot = NULL;
00111     new->dash = NULL;
00112     return new;
00113 }
00114
00116 char decode_morse_char(Node *start, char *code) {
00117     int len = strlen(code);
00118     Node *current = start;
00119     for (int i = 0; i < len; ++i) {
00120         switch (code[i]) {
00121             case '.':
00122                 current = current->dot;
00123                 break;
00124             case '-':
00125                 current = current->dash;
00126                 break;
00127             default:
00128                 // TODO print proper error
00129                 // printf("error: decode_morse_char code[%d]: %c\n", i, code[i]);
00130                 print_error_and_exit(5, "Sikertelen forditas hanyos kodtabla miatt");
00131                 break;
00132         }
00133     }

```

```

00134 // TODO add error case if current is NULL pointer
00135 return current->character;
00136 }
00137
00139 void add_generate_morse(Node **start, const char *code, const char character) {
00140     int len = strlen(code);
00141     Node *current = *start;
00142     for (int i = 0; i < len; ++i) {
00143         switch (code[i]) {
00144             case '.':
00145                 try_add_node(&current->dot);
00146                 current = current->dot;
00147                 break;
00148             case '-':
00149                 try_add_node(&current->dash);
00150                 current = current->dash;
00151                 break;
00152             default:
00153                 // TODO: print error here in a proper manner
00154                 printf("error add_generate_morse\n");
00155                 print_error_and_exit(2, "Nem ASCII karakter van a kodtablaban vagy a "
00156                                     "kodolando/dekodolando szovegben");
00157                 break;
00158         }
00159     }
00160     current->character = character;
00161 }
00162
00164 void try_add_node(Node **node) {
00165     if (*node != NULL)
00166         return;
00167     *node = init_node();
00168 }
00169
00171 void free_tree(Node *start) {
00172     if (start == NULL)
00173         return;
00174     free_tree(start->dot);
00175     free(start->dot);
00176     free_tree(start->dash);
00177     free(start->dash);
00178 }
00179
00181 void dump_tree(Node *start) { dump_node(start, 0); }
00182
00184 void dump_node(Node *start, int counter) {
00185     if (start == NULL)
00186         return;
00187     printf("%c\n", start->character);
00188     ++counter;
00189     dump_node(start->dot, counter);
00190     dump_node(start->dash, counter);
00191 }
00192
00194 void print_error_and_exit(const int error_code, char *description) {
00195     printf("Hiba: %d - %s\n", error_code, description);
00196     exit(1);
00197 }

```

6.9 D:/bme/prog1/nagyhazi/src/morse.h File Reference

```

#include "split_str.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Data Structures

- struct [Node](#)

Typedefs

- typedef struct Node [Node](#)

Functions

- char * [encode_text](#) (Node *start, char *text)
- char * [encode_word](#) (Node *start, char *word)
- char * [encode_char](#) (Node *start, char character)
- char * [decode_morse_text](#) (Node *start, char *morse_text)
- char * [decode_morse_word](#) (Node *start, char *morse_word)
- char [decode_morse_char](#) (Node *start, char *code)
- Node * [init_node](#) (void)
- void [try_add_node](#) (Node **node)
- void [add_generate_morse](#) (Node **start, const char *code, char character)
- void [free_tree](#) (Node *start)
- void [dump_tree](#) (Node *start)
- void [dump_node](#) (Node *start, int counter)
- void [print_error_and_exit](#) (int error_code, char *description)

6.9.1 Typedef Documentation

6.9.1.1 Node

```
typedef struct Node Node
```

6.9.2 Function Documentation

6.9.2.1 add_generate_morse()

```
void add_generate_morse (
    Node ** start,
    const char * code,
    const char character)
```

Generates morse code nodes in the tree based on a morse code pattern Traverses the tree following the dot (.) and dash (-) characters in the code Creates new nodes as needed and stores the character at the final position

Parameters

<i>start</i>	Pointer to the root node of the morse code tree
<i>code</i>	Morse code pattern string containing dots and dashes
<i>character</i>	Character to be stored at the end of the morse code path

Generates morse code nodes in the tree based on a morse code pattern Traverses the tree following the dot (.) and dash (-) characters in the code Creates new nodes as needed and stores the character at the final position

Parameters

<i>start</i>	Pointer to the root node of the morse code tree
<i>code</i>	Morse code pattern string containing dots and dashes

<i>character</i>	Character to be stored at the end of the morse code path
------------------	--

Definition at line 139 of file [morse.c](#).

6.9.2.2 decode_morse_char()

```
char decode_morse_char (  
    Node * start,  
    char * code)
```

Decodes a single morse code character into its plain text representation Traverses the morse tree following the dot (.) and dash (-) pattern

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>code</i>	Morse code string containing dots and dashes

Returns

Decoded character, or null character if morse code not found in tree

Decodes a single morse code character into its plain text representation Traverses the morse tree following the dot (.) and dash (-) pattern

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>code</i>	Morse code string containing dots and dashes

Returns

Decoded character, or null character if morse code not found in tree

Definition at line 116 of file [morse.c](#).

6.9.2.3 decode_morse_text()

```
char * decode_morse_text (  
    Node * start,  
    char * morse_text)
```

Decodes a complete morse code text string back into plain text Morse words in the input are separated by tabs and decoded individually The decoded words are joined with spaces in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_text</i>	Morse code text string to be decoded

Returns

Decoded plain text string with words separated by spaces, or NULL on error

Decodes a complete morse code text string back into plain text Morse words in the input are separated by tabs and decoded individually The decoded words are joined with spaces in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_text</i>	Morse code text string to be decoded

Returns

Decoded plain text string with words separated by spaces, or NULL on error

Definition at line 82 of file [morse.c](#).

6.9.2.4 decode_morse_word()

```
char * decode_morse_word (  
    Node * start,  
    char * morse_word)
```

Decodes a morse code word back into a plain text word Individual morse characters within the word are separated by spaces

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_word</i>	Morse code word string to be decoded

Returns

Decoded plain text word, or NULL on error

Decodes a morse code word back into a plain text word Individual morse characters within the word are separated by spaces

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>morse_word</i>	Morse code word string to be decoded

Returns

Decoded plain text word, or NULL on error

Definition at line 96 of file [morse.c](#).

6.9.2.5 dump_node()

```
void dump_node (
    Node * start,
    int counter)
```

Recursively prints morse tree nodes with indentation based on depth Helper function for dump_tree that handles the recursive traversal

Parameters

<i>start</i>	Current node being printed
<i>counter</i>	Indentation level counter for formatting output

Recursively prints morse tree nodes with indentation based on depth Helper function for dump_tree that handles the recursive traversal

Parameters

<i>start</i>	Current node being printed
<i>counter</i>	Indentation level counter for formatting output

Definition at line 184 of file [morse.c](#).

6.9.2.6 dump_tree()

```
void dump_tree (
    Node * start)
```

Prints a visual representation of the morse code tree structure Useful for debugging and verifying the contents of the tree

Parameters

<i>start</i>	Root node of the tree to be displayed
--------------	---------------------------------------

Prints a visual representation of the morse code tree structure Useful for debugging and verifying the contents of the tree

Parameters

<i>start</i>	Root node of the tree to be displayed
--------------	---------------------------------------

Definition at line 181 of file [morse.c](#).

6.9.2.7 encode_char()

```
char * encode_char (  
    Node * start,  
    char character)
```

Encodes a single character into its morse code representation Recursively traverses the morse tree to find the character and returns its path

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>character</i>	Single character to be encoded into morse code

Returns

Morse code string (dots and dashes) representing the character, or NULL if not found

Encodes a single character into its morse code representation Recursively traverses the morse tree to find the character and returns its path

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>character</i>	Single character to be encoded into morse code

Returns

Morse code string (dots and dashes) representing the character, or NULL if not found

Definition at line 40 of file [morse.c](#).

6.9.2.8 encode_text()

```
char * encode_text (  
    Node * start,  
    char * text)
```

Encodes a complete text string into morse code by encoding each word separately Words in the input text are separated by spaces and encoded individually The encoded words are joined with tab characters in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>text</i>	Text string to be encoded into morse code

Returns

Encoded morse code string with words separated by tabs, or NULL on error

Encodes a complete text string into morse code by encoding each word separately Words in the input text are separated by spaces and encoded individually The encoded words are joined with tab characters in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>text</i>	Text string to be encoded into morse code

Returns

Encoded morse code string with words separated by tabs, or NULL on error

Definition at line 8 of file [morse.c](#).

6.9.2.9 encode_word()

```
char * encode_word (  
    Node * start,  
    char * word)
```

Encodes a single word into morse code by encoding each character Characters within the word are encoded and separated by spaces in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>word</i>	Word string to be encoded into morse code

Returns

Encoded morse code string with characters separated by spaces, or NULL on error

Encodes a single word into morse code by encoding each character Characters within the word are encoded and separated by spaces in the output

Parameters

<i>start</i>	Root node of the morse code binary tree
<i>word</i>	Word string to be encoded into morse code

Returns

Encoded morse code string with characters separated by spaces, or NULL on error

Definition at line 22 of file [morse.c](#).

6.9.2.10 free_tree()

```
void free_tree (  
    Node * start)
```

Recursively frees all memory allocated for a morse code tree Performs depth-first traversal to deallocate all nodes in both branches

Parameters

<i>start</i>	Root node of the morse tree to be freed
--------------	---

Recursively frees all memory allocated for a morse code tree Performs depth-first traversal to deallocate all nodes in both branches

Parameters

<i>start</i>	Root node of the morse tree to be freed
--------------	---

Definition at line 171 of file [morse.c](#).

6.9.2.11 init_node()

```
Node * init_node (  
    void )
```

Initializes a new morse tree node with default values Allocates memory for a [Node](#) and sets all fields to empty/NULL values The character is set to null terminator and both pointers are set to NULL

Returns

Pointer to newly allocated and initialized [Node](#) structure

Initializes a new morse tree node with default values Allocates memory for a [Node](#) and sets all fields to empty/NULL values The character is set to null terminator and both pointers are set to NULL

Returns

Pointer to newly allocated and initialized [Node](#) structure

Definition at line 107 of file [morse.c](#).

6.9.2.12 `print_error_and_exit()`

```
void print_error_and_exit (
    const int error_code,
    char * description)
```

Prints an error message to standard output and terminates the program Used for critical errors that prevent program continuation

Parameters

<i>error_code</i>	Numeric error code identifying the error type
<i>description</i>	Human-readable error description message

Prints an error message to standard output and terminates the program Used for critical errors that prevent program continuation

Parameters

<i>error_code</i>	Numeric error code identifying the error type
<i>description</i>	Human-readable error description message

Definition at line 194 of file [morse.c](#).

6.9.2.13 `try_add_node()`

```
void try_add_node (
    Node ** node)
```

Attempts to allocate a new node if the target pointer is NULL If the pointer already points to allocated memory, the function does nothing Used as a helper to safely create child nodes during tree construction

Parameters

<i>node</i>	Pointer to a node pointer that may need allocation
-------------	--

Attempts to allocate a new node if the target pointer is NULL If the pointer already points to allocated memory, the function does nothing Used as a helper to safely create child nodes during tree construction

Parameters

<i>node</i>	Pointer to a node pointer that may need allocation
-------------	--

Definition at line 164 of file [morse.c](#).

6.10 morse.h

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/13/2025.
00003 //
00004
00005 #ifndef NAGYHAZI_MORSE_H
00006 #define NAGYHAZI_MORSE_H
00007
00008 #ifdef DEBUG_MALLOC
00009 #include "debugmalloc.h"
00010 #endif // DEBUG_MALLOC
00011
00012 #include "split_str.h"
00013
00014 #include <stdio.h>
00015 #include <stdlib.h>
00016 #include <string.h>
00017
00018 typedef struct Node {
00019     char character;
00020     struct Node *dot;
00021     struct Node *dash;
00022 } Node;
00023
00024
00033 char *encode_text(Node *start, char *text);
00034
00042 char *encode_word(Node *start, char *word);
00043
00051 char *encode_char(Node *start, char character);
00052
00061 char *decode_morse_text(Node *start, char *morse_text);
00062
00070 char *decode_morse_word(Node *start, char *morse_word);
00071
00079 char *decode_morse_char(Node *start, char *code);
00080
00081
00088 Node *init_node(void);
00089
00096 void try_add_node(Node **node);
00097
00106 void add_generate_morse(Node **start, const char *code, char character);
00107
00113 void free_tree(Node *start);
00114
00120 void dump_tree(Node *start);
00121
00128 void dump_node(Node *start, int counter);
00129
00136 void print_error_and_exit(int error_code, char *description);
00137
00138 #endif // NAGYHAZI_MORSE_H

```

6.11 D:/bme/prog1/nagyhazi/src/split_str.c File Reference

```
#include "split_str.h"
```

Functions

- [Split_str * split_str](#) (const char *arr, char delim)
- void [free_split_str](#) (Split_str *split_str)
- char * [flatten_split_str](#) (Split_str *split_str, char *delim)

6.11.1 Function Documentation

6.11.1.1 flatten_split_str()

```
char * flatten_split_str (
    Split_str * split_str,
    char * delim)
```

Flattens a [Split_str](#) structure back into a single string Joins all parts together with the specified delimiter string between them The delimiter is not added after the final part

Parameters

<i>split_str</i>	Split_str structure containing parts to be flattened
<i>delim</i>	Delimiter string to insert between parts

Returns

Flattened string with parts separated by the delimiter

Definition at line 53 of file [split_str.c](#).

6.11.1.2 free_split_str()

```
void free_split_str (
    Split_str * split_str)
```

Frees all memory allocated for a [Split_str](#) structure Deallocates each individual string in the array and the array itself

Parameters

<i>split_str</i>	Split_str structure to be freed
------------------	---

Definition at line 45 of file [split_str.c](#).

6.11.1.3 split_str()

```
Split_str * split_str (
    const char * arr,
    char delim)
```

Splits a string into an array of substrings based on a delimiter character Creates a [Split_str](#) structure containing the array of parts and their count

Parameters

<i>arr</i>	String to be split into parts
<i>delim</i>	Delimiter character used to identify split points

Returns

[Split_str](#) structure containing the split parts and their count

Definition at line 8 of file [split_str.c](#).

6.12 split_str.c

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/25/2025.
00003 //
00004
00005 #include "split_str.h"
00006
00008 Split_str *split_str(const char *arr, char delim) {
00009     int ct_len = 0;
00010     int ct_delim = 0;
00011     while (arr[ct_len] != '\0') {
00012         if (arr[ct_len] == delim)
00013             ++ct_delim;
00014         ++ct_len;
00015     }
00016
00017     int ct_split_arr = 0;
00018     int ct_split_arr_len = ct_delim + 1;
00019     char **split_arr = malloc(ct_split_arr_len * sizeof(char *));
00020
00021     int ct_before_delim = 0;
00022     for (int i = 0; i <= ct_len; ++i) {
00023         if (arr[i] == delim || ct_len == i) {
00024             // Allocate and assign value for split_arr 2d array
00025             split_arr[ct_split_arr] =
00026                 (char *)malloc(ct_before_delim + 1 * sizeof(char));
00027             for (int j = 0; j < ct_before_delim; ++j)
00028                 split_arr[ct_split_arr][j] = arr[(i - ct_before_delim) + j];
00029             split_arr[ct_split_arr][ct_before_delim] = '\0';
00030
00031             ct_before_delim = 0;
00032             ++ct_split_arr;
00033         } else {
00034             ++ct_before_delim;
00035         }
00036     }
00037
00038     Split_str *new = malloc(sizeof(Split_str));
00039     new->len = ct_split_arr;
00040     new->arr = split_arr;
00041     return new;
00042 }
00043
00045 void free_split_str(Split_str *split_str) {
00046     for (int i = 0; i < split_str->len; ++i)
00047         free(split_str->arr[i]);
00048     free(split_str->arr);
00049     free(split_str);
00050 }
00051
00053 char *flatten_split_str(Split_str *split_str, char *delim) {
00054     int ct_char = 1 + split_str->len;
00055     for (int i = 0; i < split_str->len; ++i)
00056         ct_char += strlen(split_str->arr[i]);
00057
00058     char *flattened = malloc(ct_char * sizeof(char));
00059     flattened[0] = '\0';
00060
00061     for (int i = 0; i < split_str->len; ++i) {
00062         strcat(flattened, split_str->arr[i]);
00063         if (i != split_str->len - 1)
00064             strcat(flattened, delim);
00065     }
00066
00067     return flattened;
00068 }

```

6.13 D:/bme/prog1/nagyhazi/src/split_str.h File Reference

```
#include <stdlib.h>
#include <string.h>
```

Data Structures

- struct [Split_str](#)

Typedefs

- typedef struct Split_str [Split_str](#)

Functions

- [Split_str * split_str](#) (const char *arr, char delim)
- void [free_split_str](#) ([Split_str *split_str](#))
- char * [flatten_split_str](#) ([Split_str *split_str](#), char *delim)

6.13.1 Typedef Documentation

6.13.1.1 Split_str

```
typedef struct Split_str Split_str
```

6.13.2 Function Documentation

6.13.2.1 flatten_split_str()

```
char * flatten_split_str (
    Split\_str \* split\_str,
    char * delim)
```

Flattens a [Split_str](#) structure back into a single string Joins all parts together with the specified delimiter string between them The delimiter is not added after the final part

Parameters

<i>split_str</i>	Split_str structure containing parts to be flattened
<i>delim</i>	Delimiter string to insert between parts

Returns

Flattened string with parts separated by the delimiter

Generated by Doxygen

Flattens a [Split_str](#) structure back into a single string Joins all parts together with the specified delimiter string between them The delimiter is not added after the final part

Parameters

<i>split_str</i>	Split_str structure containing parts to be flattened
<i>delim</i>	Delimiter string to insert between parts

Returns

Flattened string with parts separated by the delimiter

Definition at line 53 of file [split_str.c](#).

6.13.2.2 free_split_str()

```
void free_split_str (
    Split\_str * split_str)
```

Frees all memory allocated for a [Split_str](#) structure Deallocates each individual string in the array and the array itself

Parameters

<i>split_str</i>	Split_str structure to be freed
------------------	---

Frees all memory allocated for a [Split_str](#) structure Deallocates each individual string in the array and the array itself

Parameters

<i>split_str</i>	Split_str structure to be freed
------------------	---

Definition at line 45 of file [split_str.c](#).

6.13.2.3 split_str()

```
Split\_str * split_str (
    const char * arr,
    char delim)
```

Splits a string into an array of substrings based on a delimiter character Creates a [Split_str](#) structure containing the array of parts and their count

Parameters

<i>arr</i>	String to be split into parts
<i>delim</i>	Delimiter character used to identify split points

Returns

[Split_str](#) structure containing the split parts and their count

Splits a string into an array of substrings based on a delimiter character Creates a [Split_str](#) structure containing the array of parts and their count

Parameters

<i>arr</i>	String to be split into parts
<i>delim</i>	Delimiter character used to identify split points

Returns

[Split_str](#) structure containing the split parts and their count

Definition at line 8 of file [split_str.c](#).

6.14 split_str.h

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/25/2025.
00003 //
00004
00005 #ifndef NAGYHAZI_SPLIT_STR_H
00006 #define NAGYHAZI_SPLIT_STR_H
00007
00008 #ifdef DEBUG_MALLOC
00009 #include "debugmalloc.h"
00010 #endif // DEBUG_MALLOC
00011
00012 #include <stdlib.h>
00013 #include <string.h>
00014
00015 typedef struct Split_str {
00016     int len;
00017     char **arr;
00018 } Split_str;
00019
00027 Split_str *split_str(const char *arr, char delim);
00028
00034 void free_split_str(Split_str *split_str);
00035
00044 char *flatten_split_str(Split_str *split_str, char *delim);
00045
00046 #endif // NAGYHAZI_SPLIT_STR_H

```

6.15 D:/bme/prog1/nagyhazi/src/tests.c File Reference

```
#include "tests.h"
```

Functions

- void [run_all_tests](#) (void)
- [Node](#) * [mock_tree](#) (void)
- [Split_str](#) * [mock_split_str](#) (void)
- void [test_encode_text](#) (void)
- void [test_encode_word](#) (void)
- void [test_encode_char](#) (void)
- void [test_decode_text](#) (void)
- void [test_decode_word](#) (void)
- void [test_decode_char](#) (void)

- void [test_split_str](#) (void)
- void [test_free_split_str](#) (void)
- void [test_flatten_split_str](#) (void)
- void [test_init_node](#) (void)
- void [test_add_generate_morse](#) (void)
- void [test_free_tree](#) (void)
- void [test_file_to_morse_tree](#) (void)
- void [test_write_to_file](#) (void)
- void [test_read_all_lines](#) (void)
- void [test_remove_new_lines_chars](#) (void)

6.15.1 Function Documentation

6.15.1.1 [mock_split_str\(\)](#)

```
Split_str * mock_split_str (  
    void )
```

Definition at line [62](#) of file [tests.c](#).

6.15.1.2 [mock_tree\(\)](#)

```
Node * mock_tree (  
    void )
```

Definition at line [32](#) of file [tests.c](#).

6.15.1.3 [run_all_tests\(\)](#)

```
void run_all_tests (  
    void )
```

Definition at line [7](#) of file [tests.c](#).

6.15.1.4 [test_add_generate_morse\(\)](#)

```
void test_add_generate_morse (  
    void )
```

Definition at line [187](#) of file [tests.c](#).

6.15.1.5 [test_decode_char\(\)](#)

```
void test_decode_char (  
    void )
```

Definition at line [132](#) of file [tests.c](#).

6.15.1.6 test_decode_text()

```
void test_decode_text (  
    void )
```

Definition at line 110 of file [tests.c](#).

6.15.1.7 test_decode_word()

```
void test_decode_word (  
    void )
```

Definition at line 121 of file [tests.c](#).

6.15.1.8 test_encode_char()

```
void test_encode_char (  
    void )
```

Definition at line 99 of file [tests.c](#).

6.15.1.9 test_encode_text()

```
void test_encode_text (  
    void )
```

Definition at line 77 of file [tests.c](#).

6.15.1.10 test_encode_word()

```
void test_encode_word (  
    void )
```

Definition at line 88 of file [tests.c](#).

6.15.1.11 test_file_to_morse_tree()

```
void test_file_to_morse_tree (  
    void )
```

Definition at line 205 of file [tests.c](#).

6.15.1.12 test_flatten_split_str()

```
void test_flatten_split_str (  
    void )
```

Definition at line 167 of file [tests.c](#).

6.15.1.13 test_free_split_str()

```
void test_free_split_str (  
    void )
```

Definition at line 160 of file [tests.c](#).

6.15.1.14 test_free_tree()

```
void test_free_tree (  
    void )
```

Definition at line 198 of file [tests.c](#).

6.15.1.15 test_init_node()

```
void test_init_node (  
    void )
```

Definition at line 178 of file [tests.c](#).

6.15.1.16 test_read_all_lines()

```
void test_read_all_lines (  
    void )
```

Definition at line 243 of file [tests.c](#).

6.15.1.17 test_remove_new_lines_chars()

```
void test_remove_new_lines_chars (  
    void )
```

Definition at line 258 of file [tests.c](#).

6.15.1.18 test_split_str()

```
void test_split_str (  
    void )
```

Definition at line 145 of file [tests.c](#).

6.15.1.19 test_write_to_file()

```
void test_write_to_file (  
    void )
```

Definition at line 227 of file [tests.c](#).

6.16 tests.c

[Go to the documentation of this file.](#)

```

00001 //
00002 // Created by User01 on 11/17/2025.
00003 //
00004
00005 #include "tests.h"
00006
00007 void run_all_tests(void) {
00008     test_encode_text();
00009     test_encode_word();
00010     test_encode_char();
00011
00012     test_decode_text();
00013     test_decode_word();
00014     test_decode_char();
00015
00016     test_split_str();
00017     test_free_split_str();
00018     test_flatten_split_str();
00019
00020     test_init_node();
00021     test_add_generate_morse();
00022     test_free_tree();
00023
00024     test_file_to_morse_tree();
00025     test_write_to_file();
00026     test_read_all_lines();
00027     test_remove_new_lines_chars();
00028
00029     printf("All tests passed!\n");
00030 }
00031
00032 Node *mock_tree(void) {
00033     Node *root = malloc(sizeof(Node));
00034     root->character = '\0';
00035
00036     // a: .-
00037     root->dot = malloc(sizeof(Node));
00038     root->dot->character = '\0';
00039     root->dot->dash = malloc(sizeof(Node));
00040     root->dot->dash->character = 'a';
00041     root->dot->dash->dot = NULL;
00042     root->dot->dash->dash = NULL;
00043
00044     // b: ..
00045     root->dot->dot = malloc(sizeof(Node));
00046     root->dot->dot->character = 'b';
00047     root->dot->dot->dot = NULL;
00048     root->dot->dot->dash = NULL;
00049
00050     // c: --
00051     root->dash = malloc(sizeof(Node));
00052     root->dash->character = '\0';
00053     root->dash->dot = NULL;
00054     root->dash->dash = malloc(sizeof(Node));
00055     root->dash->dash->character = 'c';
00056     root->dash->dash->dot = NULL;
00057     root->dash->dash->dash = NULL;
00058
00059     return root;
00060 }
00061
00062 Split_str *mock_split_str(void) {
00063     Split_str *mock = malloc(sizeof(Split_str));
00064     assert(mock != NULL);
00065     mock->len = 3;
00066     mock->arr = malloc(mock->len * sizeof(char *));
00067     assert(mock->arr != NULL);
00068     mock->arr[0] = malloc((strlen("a") + 1) * sizeof(char));
00069     mock->arr[1] = malloc((strlen("b") + 1) * sizeof(char));
00070     mock->arr[2] = malloc((strlen("c") + 1) * sizeof(char));
00071     strcpy(mock->arr[0], "a");
00072     strcpy(mock->arr[1], "b");
00073     strcpy(mock->arr[2], "c");
00074     return mock;
00075 }
00076
00077 void test_encode_text(void) {
00078     Node *tree = mock_tree();
00079     assert(tree != NULL);
00080     char *encoded = encode_text(tree, "a b c");
00081     assert(encoded != NULL);
00082     assert(strcmp(encoded, ".-\t..\t--") == 0);

```

```

00083     free_tree(tree);
00084     free(tree);
00085     free(encoded);
00086 }
00087
00088 void test_encode_word(void) {
00089     Node *tree = mock_tree();
00090     assert(tree != NULL);
00091     char *encoded = encode_word(tree, "abc");
00092     assert(encoded != NULL);
00093     assert(strcmp(encoded, ".- .. --") == 0);
00094     free_tree(tree);
00095     free(tree);
00096     free(encoded);
00097 }
00098
00099 void test_encode_char(void) {
00100     Node *tree = mock_tree();
00101     assert(tree != NULL);
00102     char *encoded_a = encode_char(tree, 'a');
00103     assert(encoded_a != NULL);
00104     assert(strcmp(encoded_a, ".-") == 0);
00105     free_tree(tree);
00106     free(tree);
00107     free(encoded_a);
00108 }
00109
00110 void test_decode_text(void) {
00111     Node *tree = mock_tree();
00112     assert(tree != NULL);
00113     char *decoded = decode_morse_text(tree, ".-\t..\t--");
00114     assert(decoded != NULL);
00115     assert(strcmp(decoded, "a b c") == 0);
00116     free_tree(tree);
00117     free(tree);
00118     free(decoded);
00119 }
00120
00121 void test_decode_word(void) {
00122     Node *tree = mock_tree();
00123     assert(tree != NULL);
00124     char *decoded = decode_morse_word(tree, ".- .. --");
00125     assert(decoded != NULL);
00126     assert(strcmp(decoded, "abc") == 0);
00127     free_tree(tree);
00128     free(tree);
00129     free(decoded);
00130 }
00131
00132 void test_decode_char(void) {
00133     Node *tree = mock_tree();
00134     assert(tree != NULL);
00135     char decoded_a = decode_morse_char(tree, ".-");
00136     assert(decoded_a == 'a');
00137     char decoded_b = decode_morse_char(tree, "..");
00138     assert(decoded_b == 'b');
00139     char decoded_c = decode_morse_char(tree, "--");
00140     assert(decoded_c == 'c');
00141     free_tree(tree);
00142     free(tree);
00143 }
00144
00145 void test_split_str(void) {
00146     Split_str *result = split_str("a b c", ' ');
00147     assert(result != NULL);
00148     Split_str *expected = mock_split_str();
00149     assert(expected != NULL);
00150
00151     assert(result->len == expected->len);
00152     assert(strcmp(result->arr[0], expected->arr[0]) == 0);
00153     assert(strcmp(result->arr[1], expected->arr[1]) == 0);
00154     assert(strcmp(result->arr[2], expected->arr[2]) == 0);
00155
00156     free_split_str(result);
00157     free_split_str(expected);
00158 }
00159
00160 void test_free_split_str(void) {
00161     Split_str *s = split_str("test data split", ' ');
00162     assert(s != NULL);
00163     assert(s->len == 3);
00164     free_split_str(s);
00165 }
00166
00167 void test_flatten_split_str(void) {
00168     Split_str *s = split_str("a b c", ' ');
00169     assert(s != NULL);

```

```

00170     assert(s->len == 3);
00171     char *flattened = flatten_split_str(s, "-");
00172     assert(flattened != NULL);
00173     assert(strcmp(flattened, "a-b-c") == 0);
00174     free(flattened);
00175     free_split_str(s);
00176 }
00177
00178 void test_init_node(void) {
00179     Node *n = init_node();
00180     assert(n != NULL);
00181     assert(n->character == '\0');
00182     assert(n->dot == NULL);
00183     assert(n->dash == NULL);
00184     free(n);
00185 }
00186
00187 void test_add_generate_morse(void) {
00188     Node *root = init_node();
00189     assert(root != NULL);
00190     add_generate_morse(&root, "-.", 'a');
00191     assert(root->dot != NULL);
00192     assert(root->dot->dash != NULL);
00193     assert(root->dot->dash->character == 'a');
00194     free_tree(root);
00195     free(root);
00196 }
00197
00198 void test_free_tree(void) {
00199     Node *tree = mock_tree();
00200     assert(tree != NULL);
00201     free_tree(tree);
00202     free(tree);
00203 }
00204
00205 void test_file_to_morse_tree(void) {
00206     const char *filename = "./test_codetable.txt";
00207     FILE *f = fopen(filename, "w");
00208     assert(f != NULL);
00209     fprintf(f, "a .-\n");
00210     fprintf(f, "b ..\n");
00211     fprintf(f, "c --");
00212     fclose(f);
00213
00214     Node *root = init_node();
00215     file_to_morse_tree(&root, (char *)filename);
00216     assert(root != NULL);
00217
00218     assert(decode_morse_char(root, "-.") == 'a');
00219     assert(decode_morse_char(root, "..") == 'b');
00220     assert(decode_morse_char(root, "--") == 'c');
00221
00222     free_tree(root);
00223     free(root);
00224     remove(filename);
00225 }
00226
00227 void test_write_to_file(void) {
00228     const char *filename = "./test_output.txt";
00229     remove(filename);
00230
00231     write_to_file("-.  ..  --", (char *)filename);
00232
00233     FILE *f = fopen(filename, "r");
00234     assert(f != NULL);
00235     char buffer[100] = {0};
00236     fgets(buffer, sizeof(buffer), f);
00237     fclose(f);
00238
00239     assert(strcmp(buffer, "-.  ..  --") == 0);
00240     remove(filename);
00241 }
00242
00243 void test_read_all_lines(void) {
00244     const char *filename = "./test_read_lines.txt";
00245     FILE *f = fopen(filename, "w");
00246     assert(f != NULL);
00247     fprintf(f, "a b c\n");
00248     fclose(f);
00249
00250     char *content = read_all_lines((char *)filename);
00251     assert(content != NULL);
00252     assert(strstr(content, "a b c") != NULL);
00253
00254     free(content);
00255     remove(filename);
00256 }

```

```

00257
00258 void test_remove_new_lines_chars(void) {
00259     char *test_str = malloc(64 * sizeof(char));
00260     strcpy(test_str, "a b c\ntest\r\ndata");
00261     remove_new_line_chars(&test_str);
00262
00263     assert(test_str != NULL);
00264     assert(strchr(test_str, '\n') == NULL);
00265     assert(strchr(test_str, '\r') == NULL);
00266
00267     free(test_str);
00268 }

```

6.17 D:/bme/prog1/nagyhazi/src/tests.h File Reference

```

#include "file_handling.h"
#include "morse.h"
#include "split_str.h"
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Functions

- void [run_all_tests](#) (void)
- [Node](#) * [mock_tree](#) (void)
- [Split_str](#) * [mock_split_str](#) (void)
- void [test_encode_text](#) (void)
- void [test_encode_word](#) (void)
- void [test_encode_char](#) (void)
- void [test_decode_text](#) (void)
- void [test_decode_word](#) (void)
- void [test_decode_char](#) (void)
- void [test_split_str](#) (void)
- void [test_free_split_str](#) (void)
- void [test_flatten_split_str](#) (void)
- void [test_init_node](#) (void)
- void [test_add_generate_morse](#) (void)
- void [test_free_tree](#) (void)
- void [test_file_to_morse_tree](#) (void)
- void [test_write_to_file](#) (void)
- void [test_read_all_lines](#) (void)
- void [test_remove_new_lines_chars](#) (void)

6.17.1 Function Documentation

6.17.1.1 [mock_split_str\(\)](#)

```

Split\_str * mock\_split\_str (
    void )

```

Definition at line 62 of file [tests.c](#).

6.17.1.2 mock_tree()

```
Node * mock_tree (  
    void )
```

Definition at line 32 of file [tests.c](#).

6.17.1.3 run_all_tests()

```
void run_all_tests (  
    void )
```

Definition at line 7 of file [tests.c](#).

6.17.1.4 test_add_generate_morse()

```
void test_add_generate_morse (  
    void )
```

Definition at line 187 of file [tests.c](#).

6.17.1.5 test_decode_char()

```
void test_decode_char (  
    void )
```

Definition at line 132 of file [tests.c](#).

6.17.1.6 test_decode_text()

```
void test_decode_text (  
    void )
```

Definition at line 110 of file [tests.c](#).

6.17.1.7 test_decode_word()

```
void test_decode_word (  
    void )
```

Definition at line 121 of file [tests.c](#).

6.17.1.8 test_encode_char()

```
void test_encode_char (  
    void )
```

Definition at line 99 of file [tests.c](#).

6.17.1.9 test_encode_text()

```
void test_encode_text (  
    void )
```

Definition at line 77 of file [tests.c](#).

6.17.1.10 test_encode_word()

```
void test_encode_word (  
    void )
```

Definition at line 88 of file [tests.c](#).

6.17.1.11 test_file_to_morse_tree()

```
void test_file_to_morse_tree (  
    void )
```

Definition at line 205 of file [tests.c](#).

6.17.1.12 test_flatten_split_str()

```
void test_flatten_split_str (  
    void )
```

Definition at line 167 of file [tests.c](#).

6.17.1.13 test_free_split_str()

```
void test_free_split_str (  
    void )
```

Definition at line 160 of file [tests.c](#).

6.17.1.14 test_free_tree()

```
void test_free_tree (  
    void )
```

Definition at line 198 of file [tests.c](#).

6.17.1.15 test_init_node()

```
void test_init_node (  
    void )
```

Definition at line 178 of file [tests.c](#).

6.17.1.16 test_read_all_lines()

```
void test_read_all_lines (
    void )
```

Definition at line 243 of file [tests.c](#).

6.17.1.17 test_remove_new_lines_chars()

```
void test_remove_new_lines_chars (
    void )
```

Definition at line 258 of file [tests.c](#).

6.17.1.18 test_split_str()

```
void test_split_str (
    void )
```

Definition at line 145 of file [tests.c](#).

6.17.1.19 test_write_to_file()

```
void test_write_to_file (
    void )
```

Definition at line 227 of file [tests.c](#).

6.18 tests.h

[Go to the documentation of this file.](#)

```
00001 //
00002 // Created by User01 on 11/17/2025.
00003 //
00004
00005 #ifndef NAGYHAZI_TESTS_H
00006 #define NAGYHAZI_TESTS_H
00007
00008 #ifdef DEBUG_MALLOC
00009 #include "debugmalloc.h"
00010 #endif // DEBUG_MALLOC
00011
00012 #include "file_handling.h"
00013 #include "morse.h"
00014 #include "split_str.h"
00015
00016 #include <assert.h>
00017 #include <stdio.h>
00018 #include <stdlib.h>
00019 #include <string.h>
00020
00021 void run_all_tests(void);
00022
00023 // Mock functions
00024 Node *mock_tree(void);
00025 Split_str *mock_split_str(void);
00026
00027 // Morse tests
00028 void test_encode_text(void);
```

```
00029 void test_encode_word(void);
00030 void test_encode_char(void);
00031
00032 void test_decode_text(void);
00033 void test_decode_word(void);
00034 void test_decode_char(void);
00035
00036 // String splitter tests
00037 void test_split_str(void);
00038 void test_free_split_str(void);
00039 void test_flatten_split_str(void);
00040
00041 // Test basic morse binary tree functions
00042 void test_init_node(void);
00043 void test_add_generate_morse(void);
00044 void test_free_tree(void);
00045
00046 // File Handling tests
00047 void test_file_to_morse_tree(void);
00048 void test_write_to_file(void);
00049 void test_read_all_lines(void);
00050 void test_remove_new_lines_chars(void);
00051
00052 #endif // NAGYHAZI_TESTS_H
```

Index

- add_generate_morse
 - morse.c, [19](#)
 - morse.h, [26](#)
- arr
 - Split_str, [10](#)
- character
 - Node, [9](#)
- D:/bme/prog1/nagyhazi/src Directory Reference, [7](#)
- D:/bme/prog1/nagyhazi/src/file_handling.c, [11](#), [13](#)
- D:/bme/prog1/nagyhazi/src/file_handling.h, [14](#), [16](#)
- D:/bme/prog1/nagyhazi/src/main.c, [17](#)
- D:/bme/prog1/nagyhazi/src/morse.c, [18](#), [23](#)
- D:/bme/prog1/nagyhazi/src/morse.h, [25](#), [34](#)
- D:/bme/prog1/nagyhazi/src/split_str.c, [34](#), [36](#)
- D:/bme/prog1/nagyhazi/src/split_str.h, [37](#), [39](#)
- D:/bme/prog1/nagyhazi/src/tests.c, [39](#), [43](#)
- D:/bme/prog1/nagyhazi/src/tests.h, [46](#), [49](#)
- dash
 - Node, [9](#)
- decode_morse_char
 - morse.c, [19](#)
 - morse.h, [27](#)
- decode_morse_text
 - morse.c, [19](#)
 - morse.h, [27](#)
- decode_morse_word
 - morse.c, [20](#)
 - morse.h, [28](#)
- dot
 - Node, [9](#)
- dump_node
 - morse.c, [20](#)
 - morse.h, [29](#)
- dump_tree
 - morse.c, [20](#)
 - morse.h, [29](#)
- encode_char
 - morse.c, [21](#)
 - morse.h, [30](#)
- encode_text
 - morse.c, [21](#)
 - morse.h, [30](#)
- encode_word
 - morse.c, [21](#)
 - morse.h, [31](#)
- file_handling.c
 - file_to_morse_tree, [11](#)
 - read_all_lines, [11](#)
 - remove_new_line_chars, [12](#)
 - write_to_file, [12](#)
- file_handling.h
 - file_to_morse_tree, [14](#)
 - read_all_lines, [15](#)
 - remove_new_line_chars, [15](#)
 - write_to_file, [16](#)
- file_to_morse_tree
 - file_handling.c, [11](#)
 - file_handling.h, [14](#)
- flatten_split_str
 - split_str.c, [35](#)
 - split_str.h, [37](#)
- free_split_str
 - split_str.c, [35](#)
 - split_str.h, [38](#)
- free_tree
 - morse.c, [22](#)
 - morse.h, [32](#)
- init_node
 - morse.c, [22](#)
 - morse.h, [32](#)
- len
 - Split_str, [10](#)
- main
 - main.c, [17](#)
- main.c
 - main, [17](#)
- mock_split_str
 - tests.c, [40](#)
 - tests.h, [46](#)
- mock_tree
 - tests.c, [40](#)
 - tests.h, [46](#)
- morse.c
 - add_generate_morse, [19](#)
 - decode_morse_char, [19](#)
 - decode_morse_text, [19](#)
 - decode_morse_word, [20](#)
 - dump_node, [20](#)
 - dump_tree, [20](#)
 - encode_char, [21](#)
 - encode_text, [21](#)
 - encode_word, [21](#)
 - free_tree, [22](#)

- init_node, [22](#)
- print_error_and_exit, [22](#)
- try_add_node, [23](#)
- morse.h
 - add_generate_morse, [26](#)
 - decode_morse_char, [27](#)
 - decode_morse_text, [27](#)
 - decode_morse_word, [28](#)
 - dump_node, [29](#)
 - dump_tree, [29](#)
 - encode_char, [30](#)
 - encode_text, [30](#)
 - encode_word, [31](#)
 - free_tree, [32](#)
 - init_node, [32](#)
 - Node, [26](#)
 - print_error_and_exit, [32](#)
 - try_add_node, [33](#)
- Node, [9](#)
 - character, [9](#)
 - dash, [9](#)
 - dot, [9](#)
 - morse.h, [26](#)
- print_error_and_exit
 - morse.c, [22](#)
 - morse.h, [32](#)
- read_all_lines
 - file_handling.c, [11](#)
 - file_handling.h, [15](#)
- remove_new_line_chars
 - file_handling.c, [12](#)
 - file_handling.h, [15](#)
- run_all_tests
 - tests.c, [40](#)
 - tests.h, [47](#)
- Split_str, [10](#)
 - arr, [10](#)
 - len, [10](#)
 - split_str.h, [37](#)
- split_str
 - split_str.c, [35](#)
 - split_str.h, [38](#)
- split_str.c
 - flatten_split_str, [35](#)
 - free_split_str, [35](#)
 - split_str, [35](#)
- split_str.h
 - flatten_split_str, [37](#)
 - free_split_str, [38](#)
 - Split_str, [37](#)
 - split_str, [38](#)
- test_add_generate_morse
 - tests.c, [40](#)
 - tests.h, [47](#)
- test_decode_char
 - tests.c, [40](#)
 - tests.h, [47](#)
- test_decode_text
 - tests.c, [40](#)
 - tests.h, [47](#)
- test_decode_word
 - tests.c, [41](#)
 - tests.h, [47](#)
- test_encode_char
 - tests.c, [41](#)
 - tests.h, [47](#)
- test_encode_text
 - tests.c, [41](#)
 - tests.h, [47](#)
- test_encode_word
 - tests.c, [41](#)
 - tests.h, [48](#)
- test_file_to_morse_tree
 - tests.c, [41](#)
 - tests.h, [48](#)
- test_flatten_split_str
 - tests.c, [41](#)
 - tests.h, [48](#)
- test_free_split_str
 - tests.c, [41](#)
 - tests.h, [48](#)
- test_free_tree
 - tests.c, [42](#)
 - tests.h, [48](#)
- test_init_node
 - tests.c, [42](#)
 - tests.h, [48](#)
- test_read_all_lines
 - tests.c, [42](#)
 - tests.h, [48](#)
- test_remove_new_lines_chars
 - tests.c, [42](#)
 - tests.h, [49](#)
- test_split_str
 - tests.c, [42](#)
 - tests.h, [49](#)
- test_write_to_file
 - tests.c, [42](#)
 - tests.h, [49](#)
- tests.c
 - mock_split_str, [40](#)
 - mock_tree, [40](#)
 - run_all_tests, [40](#)
 - test_add_generate_morse, [40](#)
 - test_decode_char, [40](#)
 - test_decode_text, [40](#)
 - test_decode_word, [41](#)
 - test_encode_char, [41](#)
 - test_encode_text, [41](#)
 - test_encode_word, [41](#)
 - test_file_to_morse_tree, [41](#)
 - test_flatten_split_str, [41](#)

- test_free_split_str, [41](#)
- test_free_tree, [42](#)
- test_init_node, [42](#)
- test_read_all_lines, [42](#)
- test_remove_new_lines_chars, [42](#)
- test_split_str, [42](#)
- test_write_to_file, [42](#)
- tests.h
 - mock_split_str, [46](#)
 - mock_tree, [46](#)
 - run_all_tests, [47](#)
 - test_add_generate_morse, [47](#)
 - test_decode_char, [47](#)
 - test_decode_text, [47](#)
 - test_decode_word, [47](#)
 - test_encode_char, [47](#)
 - test_encode_text, [47](#)
 - test_encode_word, [48](#)
 - test_file_to_morse_tree, [48](#)
 - test_flatten_split_str, [48](#)
 - test_free_split_str, [48](#)
 - test_free_tree, [48](#)
 - test_init_node, [48](#)
 - test_read_all_lines, [48](#)
 - test_remove_new_lines_chars, [49](#)
 - test_split_str, [49](#)
 - test_write_to_file, [49](#)
- try_add_node
 - morse.c, [23](#)
 - morse.h, [33](#)
- write_to_file
 - file_handling.c, [12](#)
 - file_handling.h, [16](#)